

LARGE LANGUAGE MODELS

Professional Beginner-to-Intermediate Handbook

A practical guide to understanding how modern LLM systems work, how to build with them, how to improve them, and how to evaluate them.

Learning outcome A first-time reader should finish this handbook able to explain the LLM pipeline from raw text to tokens to embeddings to Transformer layers to generated tokens; understand pretraining, instruction tuning, prompting, RAG, fine-tuning and evaluation; and build a small practical LLM application safely.

Level	You should understand
Foundation	Tokens, embeddings, context windows, next-token prediction, neural networks and attention
Core architecture	Transformer blocks, self-attention, multi-head attention, position information, MLP layers and residual connections
Adaptation	Prompting, instruction tuning, PEFT/LoRA, embeddings, retrieval and RAG
Engineering	APIs, local inference concepts, latency, cost, caching, structured outputs and deployment
Evaluation	Accuracy-style task metrics, semantic evaluation, hallucination checks, retrieval metrics and human evaluation

Prerequisites: basic Python is helpful. Some familiarity with machine learning and neural networks will make the material easier, but the handbook introduces the concepts from the ground up.

Contents. Learning Path

Chapter	Topic
1	What is an LLM?
2	The text-to-token pipeline
3	Embeddings and vector representations
4	The Transformer architecture
5	Self-attention, step by step
6	How an LLM is trained
7	How an LLM generates text
8	Prompting and in-context learning
9	Instruction tuning, preference tuning and alignment
10	Fine-tuning and PEFT / LoRA
11	Embeddings, vector search and RAG
12	Tool use, agents and structured outputs
13	Evaluation and hallucination control
14	LLM application architecture and deployment
15	Security, privacy and safety
16	Practical project: RAG assistant
17	Intermediate concepts and mental models
18	Cheat sheet and self-check

Study method Read chapters 1-7 for the mental model, chapters 8-12 for application building, chapters 13-15 for production quality, then build the project in chapter 16. Use chapters 17-18 as revision material.

1. What is a Large Language Model?

Start with the simplest useful mental model.

Definition

A Large Language Model (LLM) is a neural network trained on very large collections of text so that it can predict and generate sequences of tokens. In practical use, this allows the model to answer questions, summarize, translate, classify, write code, extract information, reason over supplied context, and participate in multi-step workflows.

Key idea An LLM does not store sentences in the way a database stores rows. It learns statistical patterns in language as numerical parameters. At generation time it repeatedly estimates which token should come next.

Why “large”?

- Large training datasets: books, websites, code, documents and other text sources.
- Large parameter counts: millions to billions or more learned numerical weights.
- Large computation: training requires extensive GPU/accelerator computation.
- Large capability surface: the same base model can often perform many tasks without training a new model for each one.

LLM vs traditional ML

Traditional ML	LLM
----------------	-----

Usually trained for a narrow task	Can perform many language tasks
Features are often manually selected	Learns representations automatically
Input may be tables/numbers/images	Primary interface is token sequences, often with multimodal extensions
A classifier returns one label	An LLM usually generates a sequence token by token

Where LLMs are used

- Question answering and support assistants
- Document summarization and extraction
- Code generation and debugging assistance
- Search enhancement and RAG systems
- Information classification and routing
- Translation and rewriting
- Workflow automation with tools and APIs

2. The Text-to-Token Pipeline

Why tokenization exists

Neural networks operate on numbers, not raw words. Before text enters an LLM, a tokenizer converts it into token IDs. A token may represent a full word, part of a word, punctuation, whitespace pattern, or byte-level piece depending on the tokenizer.

```
text = "Transformers are powerful!"
# Conceptual result only:
tokens = ["Transform", "ers", " are", " powerful", "!"]
token_ids = [8245, 388, 527, 8147, 0]
```

Tokenization concepts

Concept	Meaning
Vocabulary	The set of token pieces known by the tokenizer
Token ID	Integer used to represent a token
Special token	Control token such as beginning/end/padding markers
Subword tokenization	Words can be split into reusable pieces
Context window	Maximum number of tokens the model can process at once

Important Tokens are not the same as words. A short technical word, code fragment, non-English text, or unusual name may split into several tokens. Token count affects context capacity, latency and cost.

Context window

The context window is the amount of tokenized information available to the model for a single generation. It can include system instructions, conversation history, retrieved documents, tool results and the user message. When the useful information does not fit, you need summarization, retrieval, chunking or another memory strategy.

3. Embeddings and Vector Representations

What is an embedding?

An embedding is a dense vector of numbers used to represent meaning or learned features. Similar concepts tend to occupy nearby regions of vector space. In LLMs, token embeddings are internal representations of tokens; embedding models can also create vectors for sentences, paragraphs or documents for semantic search.

```
# Conceptual vector
"cat"  -> [ 0.12, -0.44, 0.83, ... ]
"kitten" -> [ 0.10, -0.40, 0.79, ... ]
"engine" -> [-0.52, 0.18, -0.11, ... ]
```

Cosine similarity

A common way to compare two embedding vectors is cosine similarity. It measures the angle between vectors rather than their absolute magnitude.

$$\text{cosine_similarity}(a,b) = (a \cdot b) / (|a| \cdot |b|)$$

Practical use Embeddings are a core component of semantic search and RAG. You embed the user query and candidate document chunks, then retrieve chunks with high similarity.

Token embeddings vs document embeddings

Token embeddings	Document / sentence embeddings
Used inside the language model	Used for semantic search, clustering and retrieval
One representation per token position	Usually one vector per text chunk
Updated through Transformer layers	Often produced by a separate embedding model

4. The Transformer Architecture

Why Transformers changed NLP

Transformers replaced the need to process every token strictly one after another as in classic recurrent networks. Their attention mechanism lets each token interact directly with other relevant tokens in the context. This supports efficient parallel training and strong long-range language modeling.

Simplified decoder-only LLM flow

1. Tokenize the input text.
2. Convert token IDs into embedding vectors.
3. Add or incorporate position information.
4. Pass representations through many Transformer blocks.
5. Each block applies self-attention, feed-forward transformations, residual connections and normalization.
6. Project the final hidden state to vocabulary logits.
7. Convert logits to probabilities and select the next token.
8. Append the token and repeat.

Core Transformer block

Component	Purpose
Self-attention	Mixes information across token positions
Multi-head attention	Learns several interaction patterns in parallel

Feed-forward / MLP	Transforms each token representation through learned nonlinear layers
Residual connection	Adds earlier representation back to stabilize optimization
Layer normalization	Controls activation scale and improves training stability
Position information	Lets the model distinguish order and distance

Mental model Attention answers “which other token positions matter for understanding this token right now?” The MLP then performs further learned transformation on the resulting representation.

5. Self-Attention, Step by Step

Query, Key and Value

For each token representation, the model learns three projections: Query (Q), Key (K) and Value (V). A query represents what a token is looking for; keys represent what each token can be matched on; values contain information that can be combined.

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) V$

Step-by-step intuition

1. Create Q, K and V vectors using learned linear projections.
2. Compare each query with all relevant keys using dot products.
3. Scale the scores by $\sqrt{d_k}$ for numerical stability.
4. Apply a mask so a decoder-only model cannot look at future tokens during causal generation.
5. Apply softmax so scores become attention weights that sum to 1.
6. Use the weights to compute a weighted sum of value vectors.

Why multi-head attention?

Instead of learning only one attention pattern, the model uses multiple heads. Different heads can specialize in different types of relationships - such as local syntax, reference resolution, formatting, or long-range dependencies. Their outputs are concatenated and projected back into the model dimension.

```
# Shape intuition, not production code
Q = X @ Wq
K = X @ Wk
V = X @ Wv
scores = (Q @ K.T) / sqrt(d_k)
weights = softmax(mask(scores))
output = weights @ V
```

6. How an LLM is Trained

Stage 1 - Pretraining

Most decoder-only LLMs are pretrained with a next-token prediction objective. The model sees a token sequence and learns to predict each next token from the tokens before it. Because this happens over extremely large corpora, the model learns syntax, facts, style patterns, code structures and many other statistical regularities.

Cross-entropy loss is commonly used. If the correct next token should receive high probability but the model assigns low probability, the loss increases. Backpropagation computes gradients and an optimizer updates model parameters.

Training loop

1. Sample and tokenize training text.
2. Create input tokens and target next tokens.
3. Run a forward pass through the model.
4. Compute cross-entropy loss.
5. Backpropagate gradients.
6. Update parameters using an optimizer such as AdamW.
7. Repeat across massive numbers of token sequences.

Important training concepts

Concept	Why it matters
Batch size	Controls how many examples contribute to each update
Learning rate	Controls update size; too high can destabilize training
Gradient accumulation	Simulates larger effective batches when memory is limited
Mixed precision	Speeds training and reduces memory use on modern accelerators
Checkpointing	Saves model state during long training runs
Validation loss	Helps detect overfitting or training problems

Reality check Training a competitive foundation LLM from scratch is usually far beyond a personal project. Most developers build on pretrained models using prompting, RAG or parameter-efficient fine-tuning.

7. How an LLM Generates Text

Autoregressive generation

Generation is iterative. The model produces probability scores for the vocabulary, chooses one token according to the decoding strategy, appends it to the context, and runs again for the next token.

Logits, probabilities and decoding

Term	Meaning
Logit	Raw unnormalized score for a vocabulary token
Softmax	Converts logits into probabilities
Temperature	Changes how flat or sharp the probability distribution is
Top-k	Restricts sampling to the k highest-scoring candidates
Top-p	Uses the smallest candidate set whose cumulative probability reaches p
Greedy decoding	Always chooses the highest-probability token

Temperature Low temperature usually makes responses more deterministic; higher temperature can increase diversity but also risk less reliable output. Temperature does not add knowledge to the model.

Why hallucinations happen

The generation objective is to produce plausible next tokens, not to guarantee truth. If the model lacks enough evidence, misunderstands the task, or is pushed outside its reliable knowledge, it can generate fluent but incorrect statements. RAG, tool use, constraints and evaluation help reduce this risk.

8. Prompting and In-Context Learning

Prompt hierarchy

Modern LLM applications often build a context containing system/developer instructions, conversation history, retrieved evidence, tool results and the latest user request. Good prompt design makes the task, constraints, output format and evidence requirements explicit.

Useful prompting patterns

Pattern	Use
Zero-shot	Ask directly without examples
Few-shot	Provide a few examples of desired input/output behavior
Role + constraints	Set domain, rules, tone and boundaries
Structured output	Request JSON or another strict schema
Retrieval-grounded prompt	Provide source passages and require answers from them
Decomposition	Break a complex task into smaller subtasks or stages

A strong prompt template

You are an assistant for <domain>.
 Task: <what to do>
 Context: <relevant evidence>
 Constraints:
 - Do not invent missing facts.
 - State uncertainty when evidence is insufficient.
 Output format:
 - <required schema or sections>
 User input: <input>

Rule Prompt engineering improves how the model uses its existing capability and context. It does not permanently teach the model new knowledge.

9. Instruction Tuning, Preference Tuning and Alignment

From base model to assistant

A base pretrained model primarily learns next-token prediction. To make it more useful as an assistant, model developers often perform supervised instruction tuning on examples of instructions and desired responses, followed by preference-based optimization that encourages preferred behavior.

Major stages

Stage	Purpose
Pretraining	Learn general language and world/code patterns from large corpora
Supervised fine-tuning (SFT)	Learn instruction-following behavior from curated demonstrations
Preference optimization	Prefer helpful, safe and higher-quality responses over weaker alternatives
Safety tuning / policy shaping	Reduce harmful, disallowed or unreliable behavior

RLHF and DPO

RLHF (Reinforcement Learning from Human Feedback) traditionally uses human preference data to train a reward model and then optimize the language model against that reward. DPO (Direct Preference Optimization) is a simpler family of methods that learns directly from preferred vs rejected response pairs without requiring a separate reinforcement-learning loop in the same form.

10. Fine-Tuning and PEFT / LoRA

When fine-tuning is appropriate

Fine-tuning changes model parameters so behavior becomes specialized. It is useful when prompt examples are not enough, when you need a stable domain style or task behavior, or when the model must learn patterns that are repeated across many requests. It is usually not the first choice for frequently changing factual knowledge - RAG is often better for that.

Full fine-tuning vs PEFT

Full fine-tuning	PEFT / LoRA
Updates most or all model weights	Updates a small number of additional/selected parameters
High compute and memory cost	Much cheaper and easier to train
Can strongly adapt behavior	Often sufficient for domain/task adaptation
Requires careful checkpoint management	Produces small adapter weights

LoRA intuition

LoRA approximates a weight update using low-rank matrices rather than updating the full large matrix directly. Conceptually, instead of learning a full ΔW , it learns $\Delta W \approx BA$ where A and B have a much smaller rank r . This reduces trainable parameters dramatically.

```
# Conceptual Hugging Face PEFT flow
from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05
)

model = get_peft_model(base_model, config)
# Continue training model on instruction/task examples.
```

Decision rule Use prompting first. Add RAG when the model needs external/updatable knowledge. Fine-tune when you need durable behavior, formatting, terminology or task adaptation.

11. Embeddings, Vector Search and RAG

What is RAG?

Retrieval-Augmented Generation (RAG) retrieves relevant external information at request time and places that evidence into the LLM context before generation. The model can then answer using knowledge that is private, domain-specific or newer than its training.

RAG pipeline

1. Collect source documents.
2. Split them into chunks.
3. Create embeddings for each chunk.
4. Store embeddings and metadata in a vector index/database.
5. Embed the user query.
6. Retrieve the most relevant chunks.
7. Optionally rerank or filter results.
8. Build a grounded prompt containing the evidence.
9. Generate the answer and, ideally, show citations to the source chunks.

Chunking matters

Chunks that are too small may lose context; chunks that are too large may dilute relevance and waste the context window. Good chunking often respects document structure - headings, paragraphs, tables or semantic sections - and may use small overlaps.

Retrieval metrics

Metric	Meaning
Recall@k	Whether relevant evidence appears in the top-k retrieved chunks
Precision@k	How much of the retrieved top-k is actually relevant
MRR	Rewards systems that rank the first relevant result highly
nDCG	Measures graded ranking quality

```
# Simplified RAG pseudocode
query_vec = embed(user_question)
chunks = vector_db.search(query_vec, top_k=5)
context = "\n\n".join(c.text for c in chunks)

prompt = f"""
Answer only from the context below.
If the answer is missing, say you do not know.

Context:
{context}

Question: {user_question}
"""
answer = llm.generate(prompt)
```

12. Tool Use, Agents and Structured Outputs

Tool calling

An LLM can be given descriptions of external functions such as search, database queries, calculators or business APIs. Instead of guessing the result, the model can choose a tool, provide structured arguments, receive the tool output, and then continue the conversation using that output.

Agent loop

1. Read the user goal and current state.
2. Choose whether to answer directly or call a tool.
3. Generate structured tool arguments.

4. Execute the tool outside the model.
5. Return the tool result to the model.
6. Repeat until the task is complete or a limit is reached.

Important An “agent” is not magic autonomy. It is usually an orchestration loop around an LLM, tools, memory/state, rules and stopping conditions. Reliability depends on tool design, permissions, validation and observability.

Structured outputs

```
# Example schema target
{
  "intent": "support_request",
  "priority": "high",
  "summary": "Customer cannot access account",
  "needs_human": true
}
```

Structured outputs are safer than parsing free-form prose when downstream software needs predictable fields. Validate generated data against a schema before using it.

13. Evaluation and Hallucination Control

Why LLM evaluation is difficult

There is rarely one universal “accuracy score” for an LLM. Evaluation depends on the task: classification, extraction, question answering, summarization, retrieval, coding, conversation quality or tool use all need different measurements.

Evaluation layers

Layer	Examples
Task correctness	Exact match, accuracy, F1, unit tests, field-level extraction accuracy
Semantic quality	Human review, rubric scoring, model-based grading with safeguards
Groundedness	Does the answer follow supplied evidence? Are citations supported?
Retrieval quality	Recall@k, precision@k, MRR, nDCG
Safety	Policy violation rate, jailbreak resistance, PII leakage checks
Operations	Latency, cost, error rate, token usage, timeout rate

Building an eval set

1. Collect representative real user tasks.
2. Include easy, average and difficult cases.
3. Include adversarial and edge cases.
4. Write expected answers or a scoring rubric.
5. Run the same set for every model/prompt/system change.
6. Track regressions before deployment.

Best practice Treat prompts, retrieval settings and model versions like code. Changes should pass a repeatable evaluation suite before production.

Hallucination reduction methods

- Use retrieval or authoritative tools for factual questions.
- Tell the model to abstain when evidence is missing.
- Require citations or evidence spans.
- Use constrained/structured output for extraction.
- Validate critical values programmatically.
- Keep temperature low for deterministic factual tasks.
- Add human review for high-impact decisions.

14. LLM Application Architecture and Deployment

Typical production architecture

1. Client/UI sends a request.
2. Application server authenticates the user and validates input.
3. Prompt builder combines instructions, history and retrieved context.
4. Optional tools/retrieval run.
5. LLM inference API or model server generates output.
6. Output is validated, filtered or transformed.
7. Response is returned and telemetry is recorded.

Training vs inference vs application orchestration

Training / fine-tuning	Inference	Application layer
Changes model parameters	Uses fixed model parameters	Manages prompts, retrieval, tools and business logic
Offline and expensive	Runs per request	Runs per user workflow
Needs datasets and checkpoints	Needs model/API endpoint	Needs security, caching, monitoring and validation

Latency and cost levers

- Use smaller models when quality is sufficient.
- Keep prompts and retrieved context concise.
- Cache repeated deterministic results.
- Stream tokens for better perceived responsiveness.
- Batch background workloads when possible.
- Use routing: simple tasks to cheaper models, difficult tasks to stronger models.
- Track input/output tokens separately because pricing and latency can differ.

Production inference example

```
# Generic API-style pseudocode
SYSTEM = "Answer concisely. Use supplied context."

def answer(question, retrieved_context):
    messages = [
        {"role": "system", "content": SYSTEM},
        {"role": "user", "content": f"Context: {retrieved_context}"}
    ]
    Question: {question}"
    ]
    result = llm.chat(messages=messages, temperature=0.2)
    return result.text
```

15. Security, Privacy and Safety

Prompt injection

Prompt injection happens when untrusted content attempts to override application instructions or manipulate tool use. This is especially important in RAG and agents because retrieved pages or documents can contain malicious instructions.

Defenses

- Treat retrieved/user content as data, not trusted instructions.
- Separate system rules from untrusted text.
- Allowlist tools and actions; use least-privilege permissions.
- Validate tool arguments and outputs.
- Require confirmation for destructive or high-impact actions.
- Do not place secrets in prompts if the model does not need them.
- Log and audit tool calls appropriately.

Privacy

Before sending data to an LLM service, understand retention, training, access controls and contractual requirements. Remove or minimize sensitive data whenever possible. For enterprise use, define a data-classification policy and decide which information is allowed in prompts, retrieval indexes and logs.

Safety and human oversight

For medical, legal, financial, security, employment or other high-impact workflows, LLM output should generally support qualified human judgment rather than silently replace it. Add review thresholds, clear uncertainty handling and escalation paths.

16. Practical Project - Build a Small RAG Assistant

Goal

Build a document-question-answering assistant that retrieves relevant passages and asks an LLM to answer only from those passages. This project teaches embeddings, chunking, retrieval, prompting and evaluation in one workflow.

Project architecture

1. Load a set of documents.
2. Split them into chunks.
3. Embed and index the chunks.
4. Receive a user question.
5. Retrieve top-k similar chunks.
6. Construct a grounded prompt.
7. Generate the answer.
8. Return answer plus source references.
9. Log the question, retrieved chunk IDs and evaluation signals.

Minimal local-style example

```
# pip install sentence-transformers faiss-cpu numpy
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
```

```
chunks = [
    "Refunds are allowed within 14 days with proof of purchase.",
    "Standard delivery normally takes 3-5 business days.",
    "Warranty coverage lasts one year from purchase date."
```

```

]

embedder = SentenceTransformer("all-MiniLM-L6-v2")
X = embedder.encode(chunks, normalize_embeddings=True)
index = faiss.IndexFlatIP(X.shape[1])
index.add(np.asarray(X, dtype="float32"))

question = "How long does delivery take?"
q = embedder.encode([question], normalize_embeddings=True)
scores, ids = index.search(np.asarray(q, dtype="float32"), k=2)

retrieved = [chunks[i] for i in ids[0]]
context = "\n".join(retrieved)
print(context)

# Next: send `context` + `question` to your chosen LLM.
# In production, keep chunk metadata so you can cite the source.

```

What to evaluate

- Did retrieval return the correct supporting chunk?
- Did the final answer stay within the supplied context?
- Did the model abstain when the document set lacked the answer?
- Were the displayed citations linked to the actual supporting chunks?
- How much latency and token cost did each question require?

Intermediate upgrade Add a reranker, metadata filters, query rewriting, conversational memory, hybrid keyword + vector retrieval, and an automated evaluation dataset.

17. Intermediate Concepts and Mental Models

Model size is not the only factor

Quality depends on architecture, training data, training recipe, context handling, post-training, retrieval, tool access and task fit. A smaller specialized model can outperform a larger general model on a narrow workflow.

Scaling and emergent-looking capability

As training compute, data and model capacity increase, many benchmark capabilities improve smoothly, while some user-visible behaviors can appear suddenly once performance crosses a useful threshold. Do not assume every larger model is automatically better for every task.

KV cache

During autoregressive generation, attention keys and values from earlier tokens can be cached so the model does not recompute them from scratch for every new token. This significantly improves decoding efficiency but consumes memory proportional to context length and model configuration.

Quantization

Quantization stores weights and sometimes activations at lower numerical precision, such as 8-bit or 4-bit representations. It can reduce memory use and improve inference speed, sometimes with a small quality tradeoff. It is common for local and edge inference.

Distillation

Knowledge distillation trains a smaller student model using signals from a stronger teacher model. The goal is to retain useful behavior while reducing inference cost and latency.

Mixture of Experts (MoE)

MoE models contain multiple expert subnetworks but activate only a subset for a given token. This can increase total parameter capacity without using all parameters for every token, though routing and systems engineering become more complex.

Long context is not perfect memory

A large context window does not guarantee the model will use every token equally well. Important evidence can be missed, retrieval quality still matters, and long prompts increase cost and latency. Good systems prioritize the most relevant information.

18. Cheat Sheet and Self-Check

Core vocabulary

Term	One-line meaning
Token	Discrete text unit used by the model
Embedding	Dense vector representation
Attention	Weighted mixing of information across token positions
Transformer	Architecture built around attention + feed-forward blocks
Context window	Maximum token sequence available to the model at once
Pretraining	Large-scale learning, usually via next-token prediction
SFT	Supervised instruction fine-tuning
LoRA	Parameter-efficient low-rank adaptation
RAG	Retrieve external evidence before generation
Hallucination	Fluent output that is unsupported or incorrect
Inference	Using a trained model to produce outputs
Agent	LLM-based orchestration loop with tools/state/rules

What should you be able to explain now?

- Why text must be tokenized before entering an LLM.
- What embeddings represent and why vector similarity is useful.
- How Q, K and V produce attention weights.
- Why causal masking is required in decoder-only next-token training.
- How next-token prediction and cross-entropy train a language model.
- How temperature, top-k and top-p affect decoding.
- The difference between prompting, RAG and fine-tuning.
- Why LoRA is cheaper than full fine-tuning.
- How to design a basic RAG pipeline and evaluate retrieval.
- Why agents need permissions, validation and stopping conditions.
- Why LLM evaluation cannot be reduced to one universal accuracy score.
- How to reduce hallucinations in factual applications.
- How inference differs from training and application orchestration.

Recommended next projects

1. Build a document RAG assistant with citations.
2. Build a structured information extractor that validates JSON against a schema.
3. Build a tool-using assistant with one read-only API and strict argument validation.
4. Fine-tune a small open model with LoRA on a narrow instruction dataset.
5. Create an evaluation suite and compare two prompts or two models on the same tasks.

Final takeaway The strongest LLM applications rarely depend on the model alone. They combine a capable model with good context, retrieval, tools, constraints, evaluation, security and observability.